Terraform ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Terraform?
Terraform is an **Infrastructure as Code (IaC)** tool used to create, manage, and modify infrastructure using configuration files.
Instead of manually creating AWS resources from the console, we define them in Terraform code and Terraform creates them automatically.
For example, we can create VPC, EC2, S3, RDS, Load Balancer, Security Groups, etc.
Simple ga gurthupettuko:
Terraform = **Infrastructure ni code tho create/manage cheyyadaniki tool.**
Eng: Terraform is an Infrastructure as Code tool used to provision and manage infrastructure through declarative configuration files.

---

2. Why do we use Terraform?
Manual ga AWS Console lo resources create chesthe time-consuming and error-prone.
Terraform allows us to define infrastructure as code, version it in Git, reuse it, and create the same infrastructure consistently across environments.
Simple ga gurthupettuko:
Manual Infrastructure ❌
Terraform Code → Automated + Repeatable Infrastructure ✅
Eng: Terraform helps us automate infrastructure provisioning, maintain consistency, version-control infrastructure, and reduce manual configuration errors.

---

3. What is Infrastructure as Code?
Infrastructure as Code means defining infrastructure using code or configuration files instead of manually creating resources.
For example, instead of manually creating an EC2 instance, we define its configuration in Terraform.
Terraform Code
↓
Terraform
↓
AWS API
↓
EC2 / VPC / S3 / RDS
Simple ga gurthupettuko:
Infrastructure = Code lo define cheyyadam.
Eng: Infrastructure as Code is the practice of defining and managing infrastructure through machine-readable configuration files instead of manual provisioning.

---

4. What is a Terraform Provider?
A Provider is the component that allows Terraform to communicate with an external platform or service.
Examples:
AWS Provider
Azure Provider
Google Cloud Provider
Kubernetes Provider
For AWS:
```
provider "aws" {
  region = "us-east-1"
}
```
Simple ga gurthupettuko:
Provider = **Terraform ki AWS tho communication bridge.**
Eng: A Terraform Provider is a plugin that enables Terraform to interact with a specific cloud platform, service, or API.

---

5. What is a Terraform Resource?
A Resource represents an infrastructure object that Terraform manages.
For example:
```
resource "aws_instance" "web" {
  ami           = "ami-xxxxx"
  instance_type = "t3.micro"
}
```
Here aws_instance represents the AWS EC2 resource.
Simple ga gurthupettuko:
Resource = **Actual infrastructure object.**
Eng: A Terraform Resource represents an infrastructure object such as an EC2 instance, VPC, S3 bucket, or security group that Terraform manages.

---

6. What is a Terraform Variable?
Variables are used to make Terraform configurations reusable and configurable.
Instead of hardcoding values:
instance_type = "t3.micro"
we can define:
variable "instance_type" {

```
 default = "t3.micro"
}
```
Then use that variable in the resource.
Simple ga gurthupettuko:
Variable = **Change ayye value ni externalize cheyyadam.**
Eng: Terraform variables allow us to parameterize configurations so the same code can be reused with different input values.

---

7. What is an Output in Terraform?
Outputs display useful information after Terraform creates infrastructure.
For example, after creating an EC2 instance, we can output its public IP.
```
output "instance_ip" {
 value = aws_instance.web.public_ip
}
```
Simple ga gurthupettuko:
Output = **Created resource nunchi useful information.**
Eng: Terraform outputs expose useful values from managed infrastructure, such as instance IP addresses, load balancer DNS names, or resource IDs.

---

8. What is Terraform Module?
A Module is a reusable collection of Terraform configuration files.
For example, instead of writing VPC configuration repeatedly, we can create one VPC module and reuse it for Dev, QA, and Production.
```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```
Simple ga gurthupettuko:
Module = **Reusable Terraform code.**
Eng: A Terraform Module is a reusable collection of configuration that helps standardize and organize infrastructure code.

---

9. Why do we use Terraform Modules?
Modules reduce duplicate code and make infrastructure easier to maintain.
For example, the same VPC module can be reused with different CIDR blocks and subnet configurations for different environments.
Simple ga gurthupettuko:
Duplicate Code ❌
Reusable Module ✅
Eng: Modules improve reusability, consistency, maintainability, and standardization of Terraform infrastructure.

---

10. What is Terraform State?
Terraform State is a file that stores information about the infrastructure Terraform manages.
It maintains the relationship between Terraform configuration and real infrastructure resources.
By default, the state file is commonly named:
terraform.tfstate
Simple ga gurthupettuko:
State = **Terraform ki existing infrastructure gurinchi information.**
Eng: Terraform State stores the mapping between Terraform configuration and the real infrastructure resources managed by Terraform.

---

11. Why is Terraform State important?
Terraform uses the state to understand what resources already exist and what changes are required.
When we run terraform plan, Terraform compares the configuration, state, and actual infrastructure to determine the required actions.
Simple ga gurthupettuko:
Code + State + Real Infrastructure → Terraform decides changes.
Eng: Terraform State is important because it helps Terraform track managed resources and determine what changes are required during planning and execution.

---

12. Where should Terraform State be stored in a real project?
For a team environment, we generally use a **remote backend** rather than keeping state only on a developer's local machine.
In AWS, a common approach is storing Terraform State in an **S3 bucket**, with appropriate controls for state access and locking/concurrency management according to the Terraform setup.
This allows multiple team members or CI/CD pipelines to work with the same state.
Simple ga gurthupettuko:
Local State ❌
Remote Backend → Team + CI/CD friendly ✅
Eng: In team environments, Terraform State is commonly stored in a secure remote backend so it can be shared and managed consistently by multiple users and CI/CD systems.

13. What is Terraform Backend?
A Backend defines **where Terraform stores its state** and how state operations are handled.
For example, an AWS-based project may use an S3-backed remote state configuration.
Backend configuration is important for team collaboration and state management.
Simple ga gurthupettuko:
Backend = **State ekkada store/manage cheyyali.**
Eng: A Terraform Backend defines how and where Terraform State is stored and managed.

---

14. What is terraform init**?**
terraform init initializes a Terraform working directory.
It downloads required providers and modules and initializes the configured backend.
Example:
terraform init
Usually, this is the first Terraform command we run in a new project.
Simple ga gurthupettuko:
terraform init = **Project setup.**
Eng: terraform init initializes the Terraform working directory by configuring the backend and installing required providers and modules.

---

15. What is terraform plan**?**
terraform plan shows what Terraform **plans to change** without actually applying those changes.
Example:
terraform plan
It may show:
+ create
~ update
- destroy
This is very important before making Production changes.
Simple ga gurthupettuko:
Plan = **What changes happen ani preview.**
Eng: terraform plan compares the desired configuration with the current state and infrastructure information and shows the proposed changes without applying them.

---

16. What is terraform apply**?**
terraform apply executes the planned infrastructure changes.
Example:
terraform apply
Terraform creates, updates, or destroys resources according to the configuration and approved plan.
Simple ga gurthupettuko:
Plan = Preview
Apply = **Actual change**
Eng: terraform apply executes the Terraform configuration and applies the required infrastructure changes.

---

17. What is terraform destroy**?**
terraform destroy removes infrastructure managed by the Terraform configuration.
Example:
terraform destroy
It should be used carefully, especially in Production.
Simple ga gurthupettuko:
Destroy = **Terraform manage chestunna resources remove cheyyadam.**
Eng: terraform destroy removes resources managed by Terraform according to the current configuration and state.

---

18. What is the Terraform workflow?
The common Terraform workflow is:
Write Code
↓
terraform init
↓
terraform validate
↓
terraform plan
↓
terraform apply
In real projects, Git and CI/CD are commonly added around this workflow.
Simple ga gurthupettuko:
Code → Init → Validate → Plan → Apply
Eng: A standard Terraform workflow involves initializing the project, validating configuration, reviewing a plan, and then applying approved infrastructure changes.

19. What is terraform validate**?**
terraform validate checks whether the Terraform configuration is syntactically and structurally valid.
Example:
terraform validate
It helps identify configuration errors before running terraform plan.
Simple ga gurthupettuko:
Validate = **Terraform code basic correctness check.**
Eng: terraform validate checks whether the Terraform configuration is valid and correctly structured.

---

20. What is Terraform Import?
Terraform Import is used to bring an **existing infrastructure resource** under Terraform management.
For example, if an EC2 instance was manually created from the AWS Console, we can import it into Terraform management.
Modern Terraform workflows use import functionality together with configuration so the resource can be properly represented in code.
Simple ga gurthupettuko:
Existing AWS Resource → Terraform Management
Eng: Terraform import allows existing infrastructure resources to be brought under Terraform management without recreating them.

---

21. Why do we need Terraform Import?
Suppose someone manually created an EC2 instance in Production.
Later, the team decides that all infrastructure should be managed using Terraform.
Instead of deleting and recreating the EC2 instance, we can import the existing resource into Terraform management and define the corresponding configuration.
Simple ga gurthupettuko:
Manual Resource → Import → Terraform Managed
Eng: Import is useful when existing manually created infrastructure needs to be brought under Terraform management without recreating the resource.

---

22. What is Terraform State Locking?
State locking prevents multiple Terraform operations from modifying the same state simultaneously.
This is important in team environments because two engineers or CI/CD pipelines should not modify the same infrastructure state at the same time.
Simple ga gurthupettuko:
One State → One operation at a time.
Eng: State locking prevents concurrent Terraform operations from corrupting or conflicting over the same state.

---

23. What happens if two developers run terraform apply **at the same time?**
If proper remote state locking is configured, one Terraform operation acquires the lock and the other operation waits or fails depending on the backend and configuration.
This prevents simultaneous state modifications.
Simple ga gurthupettuko:
Developer A → State Lock 🔒
Developer B → Wait / Fail
Eng: With proper state locking, concurrent Terraform operations are prevented from modifying the same state simultaneously.

---

24. What is Terraform Drift?
Drift occurs when the real infrastructure is changed outside Terraform.
For example, Terraform created an EC2 instance with one instance type, but someone manually changes the instance type from the AWS Console.
Now the real infrastructure differs from Terraform's expected configuration.
Simple ga gurthupettuko:
Terraform Code ≠ Real Infrastructure → **Drift**
Eng: Terraform drift occurs when infrastructure changes outside Terraform cause the actual environment to differ from the configuration Terraform expects to manage.

---

25. How do you detect Terraform Drift?
We can run:
terraform plan
Terraform compares the desired configuration with the state and information about the actual infrastructure and can show changes that need to be reconciled.
Simple ga gurthupettuko:
Manual Change → terraform plan → Drift identify
Eng: I use terraform plan to identify differences between the desired Terraform configuration and the infrastructure currently being managed.

---

26. What happens if someone manually deletes a Terraform-managed resource?
Terraform will detect that the resource is missing when the infrastructure is evaluated during planning or refresh-related operations.
If the resource is still defined in Terraform configuration, Terraform will generally propose recreating it.
Example:
Terraform Code → EC2 should exist
AWS → EC2 deleted manually

↓
terraform plan
↓
EC2 recreate required
Simple ga gurthupettuko:
Code says resource exist → AWS lo missing → Terraform proposes recreate.
Eng: If a Terraform-managed resource is deleted outside Terraform while it remains defined in configuration, Terraform can detect the difference and plan to recreate it.

---

27. What is terraform fmt**?**
terraform fmt formats Terraform configuration files according to Terraform's standard formatting style.
Example:
terraform fmt
It helps keep Terraform code consistent across developers and repositories.
Simple ga gurthupettuko:
terraform fmt = **Code formatting.**
Eng: terraform fmt automatically formats Terraform configuration files using the standard Terraform style.

---

28. What is the difference between terraform plan **and** terraform apply**?**
terraform plan only shows the proposed infrastructure changes.
terraform apply actually executes those changes.
Simple ga gurthupettuko:
Plan = **Preview**
Apply = **Execute**
Eng: terraform plan previews proposed infrastructure changes, while terraform apply executes those changes.

---

29. How do you manage Dev, QA, and Production using Terraform?
We can create reusable modules and provide environment-specific configuration.
For example:
Terraform
├── modules
│ ├── vpc
│ ├── ec2
│ └── rds
│
├── dev
├── qa
└── prod
Each environment can have different variables, state, account configuration, and resource sizing while reusing common modules.
Simple ga gurthupettuko:
Common Modules + Different Variables/State = Dev / QA / Prod
Eng: I use reusable Terraform modules with environment-specific configuration and isolated state management to maintain Dev, QA, and Production infrastructure consistently.

---

30. How do you handle Terraform secrets?
I avoid hardcoding passwords, API keys, and other sensitive values directly in Terraform code or Git.
Depending on the architecture, secrets can be retrieved from a dedicated secrets-management system or supplied securely through CI/CD and Terraform variables.
Sensitive Terraform variables and outputs should also be marked appropriately.
Simple ga gurthupettuko:
Secrets → Secret Manager / Secure CI/CD
Hardcoded Password ❌
Eng: I avoid storing secrets in Terraform source code and use secure secret-management or CI/CD mechanisms while ensuring sensitive values are not exposed unnecessarily.

---

31. What is sensitive = true **in Terraform?**
sensitive = true tells Terraform that a variable or output contains sensitive information.
For example:
```
variable "db_password" {
  type      = string
  sensitive = true
}
```
This helps prevent the value from being displayed normally in Terraform command output.
However, marking a value sensitive does **not** mean it is encrypted everywhere, so state security is still very important.
Simple ga gurthupettuko:
Sensitive = **Output lo secret ni unnecessarily show cheyyakunda.**
Eng: sensitive = true prevents sensitive values from being displayed normally in Terraform CLI output, but secure state storage is still required.

---

32. How do you handle Terraform State in a Production environment?
I would use a secure remote backend with controlled access and state locking/concurrency protection.

I would also restrict access to the state because Terraform State may contain sensitive infrastructure information and sometimes sensitive values.
The state should be backed up according to the organization's recovery requirements.
Simple ga gurthupettuko:
Production State → Remote + Secure + Restricted Access + Recovery
Eng: In Production, I use a secure remote backend, controlled access, state locking where supported, and appropriate backup and recovery practices for Terraform State.

---

33. How do you troubleshoot a Terraform deployment failure?
First I check the Terraform error message and identify which resource failed.
Then I verify provider authentication, variables, resource dependencies, IAM permissions, networking, quotas, and the cloud provider's API response.
I also run:
terraform validate
terraform plan
and review the CI/CD logs if Terraform is executed through a pipeline.
Simple ga gurthupettuko:
Error → Resource → Permissions → Variables → Dependency → Cloud API → Root Cause
Eng: I troubleshoot Terraform failures by checking the error message, provider authentication, variables, dependencies, permissions, quotas, and cloud-provider API responses.

---

34. How do you use Terraform in CI/CD?
A typical flow is:
Developer
↓
Git
↓
Pull Request
↓
Terraform fmt / Validate
↓
Terraform Plan
↓
Approval
↓
Terraform Apply
↓
AWS Infrastructure
In Production, we generally review the plan and use an approval mechanism before applying infrastructure changes.
Simple ga gurthupettuko:
Git → Validate → Plan → Approval → Apply
Eng: In CI/CD, Terraform is commonly used to validate infrastructure code, generate a plan, obtain approval where required, and apply controlled infrastructure changes.

---

35. Explain a real-time Terraform project flow.
Suppose we need to create a complete AWS application infrastructure.
We create reusable Terraform modules for VPC, subnets, Security Groups, EC2, ALB, and RDS.
Developers commit the Terraform code to Git. CI validates the code and generates a plan. After review and approval, the pipeline executes Terraform Apply and creates or updates the infrastructure.
Git
↓
Terraform Code
↓
CI Pipeline
↓
terraform fmt
↓
terraform validate
↓
terraform plan
↓
Approval
↓
terraform apply
↓
AWS Infrastructure
Simple ga gurthupettuko:
Git → Validate → Plan → Approval → Apply → AWS
Eng: In a real-time DevOps environment, Terraform code is stored in Git, validated and planned through CI/CD, reviewed through an approval process, and then applied to provision or update AWS infrastructure consistently.